

Scenario Prompts



1. Misaligned Ownership & Responsibilities

- A KPI breaks in Looker. Who investigates, who fixes, and how fast do we know if it was a pipeline bug vs. business logic misalignment?
- A stakeholder wants to request a new metric. Who do they go to? Does it end up with Analytics Engineers, Product Analysts, or somewhere else?
- An event schema in Snowplow changes. Which team owns the fix, and how do downstream teams find out?
- A dbt PR is opened that impacts core models. Who approves, and is it always the same team?
- A failed test triggers alerts. Who gets tagged, and how clear is the ownership?
- A DS team builds a new model requiring features from multiple domains. Who owns those features, and how are they shared?
- There's confusion over who owns **feature engineering code**: DE, AE, or DS? How does this architecture clarify responsibilities?
- A production ML model starts degrading due to data drift. Who is responsible for monitoring and retraining?

2. Limited Data Governance & Quality Assurance

- A downstream business user asks: “*Where does this KPI come from?*” How easily can we trace the lineage and explain the logic?
 - A dataset used by DS models is modified. Who knows what is impacted, and how?
 - Backend data is ingested without QA. How do errors get caught?
 - Documentation is weak/nonexistent. How does your architecture enforce or encourage documentation?
 - Two different dashboards show different numbers for the same metric. How do we know which is correct?
 - A model's input schema changes due to an upstream pipeline adjustment. How do we know which models are impacted, and who coordinates the fix?
 - We need to reproduce a past ML experiment. How easily can we retrieve the exact data, features, and transformations?
 - Model monitoring shows a fairness or performance issue. Who owns ongoing governance and resolution?
-

3. The Pipeline is a Monster

Scenario Prompts

- A new analyst joins. How long until they can safely contribute to dbt models without breaking things?
- We want to introduce a new KPI. How long would it take to get it visible in Looker?
- A breaking change is pushed in a central model. How do we figure out what downstream assets are affected?
- A single KPI has multiple possible sources. How do we enforce a single source of truth?
- Events are dropped due to schema mismatches. Who catches this, and how do we ensure consistency across models?
- Dependencies between models cause long lead times for even small changes. How does this architecture reduce (or worsen) that?
- Deployment of a model requires **real-time inference**. Does this architecture support it, or is it bolted on separately?
- DS teams need to scale and reuse feature pipelines. Does the architecture simplify this, or add complexity?